

“Smart Waste Management System”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

By

ILI. HARSHIKA **2520030607**

ANGELA JOSE **2520030365**

CHINTA PAVAN KUMAR 2520030312

Under the Esteemed Guidance of

Dr Spandana S

Assistant Professor

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



Declaration

The Project Report entitled “**Smart Waste Management System**” is a record of Bonafide work of **ILI. HARSHIKA – 2520030607, ANGELA JOSE – 2520030365, CHINTA PAVAN KUMAR – 2520030312** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering to the K L (Deemed to be) University. The results embodied in this report have not been copied from any other departments/University/Institute.

ILI. HARSHIKA – 2520030607

ANGELA JOSE – 2520030365

CHINTA PAVAN KUMAR – 2520030312

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

This is to certify that the project based report entitled **“Smart Waste Management System”** is a bonafide work done and submitted by **ILI. HARSHIKA – 2520030607, ANGELA JOSE – 2520030365, CHINTA PAVAN KUMAR – 2520030312** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science and Engineering, K L (Deemed to be University), during the academic year [2026-2027].

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success of this project would not have been possible without the timely help and guidance rendered by many people. We wish to express our sincere thanks to all those who assisted us in one way or the other in the completion of our project.

Our greatest appreciation goes to our Course Coordinator and our guide **Dr Spandana S**, Assistant Professor, Department of Computer Science and Engineering, whose tremendous support, encouragement and guidance cannot be expressed in words.

We express our gratitude to Head of the Department of Computer Science and Engineering, for providing us with adequate facilities, ways and means by which we were able to complete this project-based lab.

We thank all the members of the teaching and non-teaching staff who have assisted us directly or indirectly in the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

ILI. HARSHIKA – 2520030607

ANGELA JOSE – 2520030365

CHINTA PAVAN KUMAR – 2520030312

Abstract

Waste collection in many areas still depends on fixed schedules and manual monitoring. Bins often fill before the scheduled collection and overflow, while other bins are visited when they hardly need it. This leads to unnecessary trips, wastage of fuel and manpower, poorly organized records, higher operational cost, and a negative effect on the cleanliness of public and residential areas.

To address this, the project “Smart Waste Management System” presents a database-driven solution that uses simulated real-time waste-level data to monitor smart bins and manage waste collection. A Node.js program simulates the readings that sensors would send. The readings are stored in a centralized PostgreSQL database and compared with a predefined threshold. Bins that cross the threshold are identified, a collection request is created, assigned to a collector and a vehicle, and tracked until collection is completed. Waste-level and collection history is retained so that reports and analytics can be generated.

The system is organized into six modules: User & Role Management, Smart Bin Management, Waste-Level Monitoring, Collection Management, Collector & Vehicle Management, and Reports & Analytics. It is implemented with PostgreSQL for the database, Node.js and Express.js for the backend REST APIs, Postman for API testing, and Docker Desktop for containerized deployment, with Visual Studio Code as the development environment.

By basing collection on actual bin data instead of fixed schedules, the system provides a structured, data-driven approach that follows the sequence Monitor, Detect, Assign, Track and Analyze. It shows how database technology can make waste management more efficient, organized and reliable, and it provides a base that can later be extended with IoT sensors, GPS tracking, route optimization, a mobile app, real-time alerts and predictive analysis.

Index

S. No	Chapters	Topics	Page.no
		Acknowledgement Abstract	
1	Introduction	1.1 Background of the project 1.2 Problem statement 1.3 Scope of the project 1.4 Objective(s) 1.5 Importance of the application 1.6 Target users/audience	1-4
2	System Requirements	2.1 Hardware requirements 2.2 Software requirements 2.3 Development tools and frameworks	5-6
3	Technology Stack	3.1 Front-end design 3.2 Back-end design 3.3 Database design 3.4 Version control 3.5 APIs or external services	7-9
4	System Architecture	4.1 High-level architecture diagram 4.2 Description of each layer (frontend, backend, database) 4.3 Deployment architecture (cloud, local, CI/CD pipelines)	10-11
5	Design	5.1 System workflow design 5.2 ER Diagram / Database schema	12-14
6	Implementation	6.1 Module-wise implementation 6.2 Front-end logic (client interaction and data simulation) 6.3 API integration 6.4 Authentication & Authorization	15-18
7	Features	7.1 List of main features 7.2 How each feature works (user perspective + technical description)	19-20
8	Testing	8.1 Postman testing (frontend/backend) 8.2 Integration testing 8.3 Tools used for testing 8.4 Test cases and results	21-23
9	Deployment	9.1 Steps to deploy 9.2 Environment configuration 9.3 Hosting of frontend and backend	24-25

S. No	Chapters	Topics	Page.no
10	Challenges & Limitations	10.1 Issues faced during development 10.2 Solutions applied 10.3 Current limitations or known bugs	26-27
11	Future Enhancements	11.1 Planned features 11.2 Possible integrations or optimizations	28
12	Conclusion	12.1 Summary of the project 12.2 What was achieved 12.3 Skills learned during development	29-30
13	References	- Books, tutorials, APIs, documentation sites used	31
14	Appendices	- Screenshots of the app - Sample code snippets - Installation/setup instructions - User manual or guide - Geo tag photos with guide - Review forms with guide signatures	32-35

CHAPTER - 1 INTRODUCTION

1.1 Background of the Project

Solid waste generation is one of the most visible consequences of urban growth. Every household, market, office, campus and public place produces waste every day, and that waste has to be collected, transported and disposed of on a regular basis. When collection works well, streets stay clean and public health is protected. When it does not, bins overflow, waste spills onto roads, drains get blocked, and the surrounding area becomes unhygienic. Municipal bodies and campus administrations therefore spend a significant share of their operating budget on keeping collection running smoothly.

In most places, waste collection still follows fixed schedules. A vehicle is sent on a pre-planned route on certain days and at certain times, and workers empty every bin along that route whether it is full or not. Waste generation, however, is not uniform. A bin near a food market or a bus stand can fill within hours, whereas a bin in a quiet residential lane may stay half empty for days. A fixed schedule cannot respond to this variation, so some bins overflow before the vehicle arrives while others are visited when there is almost nothing to collect.

Advances in database technology, backend frameworks and sensor-based monitoring now make it possible to base collection decisions on the actual condition of each bin instead of on a calendar. If the fill level of every bin is known and stored in a central database, the system can automatically identify the bins that need attention, raise collection requests, assign them to staff and keep a permanent record of what happened.

The **Smart Waste Management System** developed in this project follows exactly this idea. It is a database-driven system that uses simulated real-time waste-level data to monitor smart bins and manage the collection process. Because installing physical sensors on a large number of bins is beyond the scope of an academic project, the readings that a sensor would normally send are generated by a Node.js simulation program. The rest of the system is built as it would be for real sensors: the readings are stored in a PostgreSQL database, compared with a predefined threshold, converted into collection requests, assigned to collectors and vehicles, tracked until completion, and finally kept as historical records from which reports are generated.

From an academic point of view, the project brings together several core areas of computer science:

- **Database Management:** designing a normalized relational schema in PostgreSQL for bins, readings, users, collectors, vehicles and collection requests.

- **Backend Development:** building REST APIs with Node.js and Express.js that connect the database to its clients.
- **Software Engineering:** dividing the system into six independent modules with clearly defined responsibilities.
- **Testing and Deployment:** verifying the APIs with Postman and packaging the application and database with Docker for consistent deployment.

To summarize, the background of this project rests on three observations: waste generation varies by location and time; fixed collection schedules do not reflect the real condition of bins; and organized, stored data can greatly improve collection planning. The project turns these observations into a working, data-driven solution.

1.2 Problem Statement

Waste collection in many areas depends on fixed schedules and manual monitoring. Workers inspect bins by hand or simply follow a planned route, and the actual waste level of a bin is often unknown until the vehicle reaches it. Basic sensors and route tools, where they exist, usually work as separate pieces rather than as one connected system, and very little organized data is available afterwards to analyze what was collected, where, and when.

This approach creates several connected problems:

- **Overflowing bins:** bins fill up before the next scheduled collection, causing litter, bad odour and unhygienic surroundings.
- **Fixed schedules:** collection is planned by time and not by the real condition of the bin.
- **Unnecessary trips:** bins that do not yet need collection are visited anyway, while others are left waiting.
- **Fuel wastage:** every unnecessary trip burns fuel and increases the operating cost.
- **Manpower wastage:** collection staff are used inefficiently because their work is not directed to where it is actually needed.
- **Poor records:** there is no organized record of waste levels and collections, which makes planning and accountability difficult.

Taken together, these problems result in inefficient waste collection. Operational cost increases and the cleanliness of public and residential areas is affected. A delayed response is also hard to avoid: when a bin fills earlier than expected, nobody finds out until someone happens to see it.

The core problem addressed by this project is therefore the lack of a single, data-driven system that (a) knows the current fill level of each bin, (b) automatically detects when a bin needs to

be emptied, (c) manages the resulting collection work from creation to completion, and (d) preserves the data for reports and future planning. The proposed system addresses these needs by combining simulated real-time waste-level data with a centralized database and a set of backend services.

1.3 Scope of the Project

The scope of the project covers the complete flow of information from a waste-level reading to a completed collection record. Within this scope the system provides the following:

- Registration and login of users with role-based access control.
- Creation and management of smart bins, including location, capacity and status.
- Generation and storage of simulated real-time waste-level readings for every bin.
- Comparison of each reading with a predefined threshold and identification of bins that require collection.
- Creation, assignment and tracking of collection requests until collection is completed.
- Management of collector details, vehicle information and their assignments.
- Maintenance of waste-level and collection history, and generation of reports such as bin-wise and location-wise analysis.

Technically, the system is built as a backend and database solution using PostgreSQL for storage, Node.js with Express.js for the REST APIs, Postman for API testing and Docker Desktop for containerized deployment. Waste-level readings are produced by a Node.js simulation program that stands in for physical sensors.

Exclusions and limitations. The current version does not use real IoT sensors, does not track vehicles by GPS, does not calculate optimized collection routes, and does not include a mobile application or automatic real-time alerts. These items are deliberately left for future work and are discussed in Chapter 11. The system also assumes that every bin has a fixed location and capacity that are entered by an administrator.

1.4 Objectives of the Project

The main objective of the project is to design and develop a database-driven system that monitors waste bins using simulated real-time data and manages waste collection based on actual bin conditions rather than fixed schedules. The specific objectives are:

1. **Monitor waste levels:** use simulated real-time data representing sensor readings to follow the fill level of every bin.

2. **Centralize data:** store waste-bin, waste-level, collector, vehicle and collection information in one database.
3. **Detect thresholds:** identify the bins that require collection by comparing readings against predefined levels.
4. **Track collection:** create, assign and track collection requests until they are completed.
5. **Maintain history:** keep waste-level and collection records and generate reports from them.
6. **Support data-driven decisions:** rely on actual bin data, and not only on fixed schedules, when planning collection.

Beyond these functional goals, the project also aims to demonstrate how relational database design, backend API development, API testing and containerization can be combined into a complete and well-structured software system.

1.5 Importance of the Application

Efficient waste collection matters to everyone who lives or works in an area. Clean surroundings reduce health risks, protect the environment and improve the quality of public spaces. For the organization that manages collection, efficiency translates directly into lower cost. The importance of this application can be understood from the following points.

- **Timely collection:** bins that cross the threshold are identified immediately, which reduces the chance of overflow.
- **Reduced unnecessary trips:** collection effort is directed to bins that actually need it, which saves fuel and travel time.
- **Better use of manpower:** requests are assigned to specific collectors, so staff time is used where it is needed.
- **Accountability:** every request carries its status and timestamps, so it is always clear what has been done and what is pending.
- **Organized historical data:** stored readings and collection records support analysis of waste patterns by bin and by location, and help plan future collection.
- **Foundation for smart-city solutions:** the same design can later be connected to real IoT sensors, GPS tracking and route optimization.

The project is also of educational value. It shows how a real-world problem can be analyzed, modelled in a relational database, implemented through APIs, tested and deployed in containers, which are the same steps followed in professional software development.

1.6 Target Users / Audience

The system is intended for the following groups of users:

- **Municipal and local-body administrators** who are responsible for keeping an area clean and who need an overall view of bins and collection activity.
- **Campus, apartment and industrial-park facility managers** who manage a limited number of bins and collection staff.
- **Waste-collection supervisors** who create, assign and monitor collection requests.
- **Collectors and vehicle drivers** who receive assigned requests and update their status.
- **Planners, researchers and students** who can use the stored waste-level and collection history for analysis, or use the project as a reference for database-driven system design.

Within the system these groups are represented by user roles, so that each person can see and do only what their role allows.

CHAPTER - 2 SYSTEM REQUIREMENTS

2.1 Hardware Requirements

Because the project is a database and backend system without any user-facing graphics, its hardware needs are modest. The requirements below are divided into those for development and testing and those for running the deployed system.

Development machine

Component	Minimum / Recommended
Processor	Intel Core i3 / AMD Ryzen 3 or higher (Core i5 / Ryzen 5 recommended)
RAM	8 GB minimum (Docker Desktop, PostgreSQL, Node.js and a browser run together); 16 GB recommended
Storage	256 GB SSD with at least 10 GB free for Docker images, containers and database volumes
Display	1366 x 768 or higher
Network	Basic internet connection for downloading packages and Docker images
Input devices	Keyboard and mouse

Deployment environment

The containerized system runs on any computer that can run Docker Desktop. A machine with a dual-core processor, 4 GB of RAM available to Docker and about 5 GB of free disk space is sufficient for the PostgreSQL container, the API container and the simulator. For larger installations, a dedicated server with more memory and storage would be required for the database, because the waste-level table grows continuously as new readings are stored.

Future hardware (not part of this version)

When the system is later connected to real smart bins, each bin would need an ultrasonic or similar fill-level sensor, a microcontroller and a communication module. Collection vehicles would need a GPS device. These are not required for the present simulated version.

2.2 Software Requirements

Software	Purpose / Version
Operating system	Windows 10/11, Ubuntu Linux or macOS
Node.js (LTS)	Runtime for the backend APIs and the data-simulation program
PostgreSQL	Relational database that stores bins, readings, users, collectors, vehicles and collection requests

Software	Purpose / Version
Express.js	Web framework used to build the REST APIs
Docker Desktop	Containerization of the application and database for consistent deployment
Postman	Testing and verification of the backend REST APIs
Visual Studio Code	Code editor used to develop and manage the project
Web browser	Google Chrome, Microsoft Edge or Firefox (for documentation, Postman web and database tools)

All the software listed is free or has a free edition for students, which keeps the project easy to reproduce on any student machine.

2.3 Development Tools and Frameworks

The following tools and frameworks were used to design, build and verify the system.

- **Visual Studio Code** is the development environment. Its integrated terminal, debugger and extensions for JavaScript, Docker and PostgreSQL allow the code, containers and database to be managed from a single window.
- **Node.js** provides a fast, event-driven JavaScript runtime. It is used both for the backend server and for the program that simulates real-time waste-level readings.
- **Express.js** is a minimal web framework for Node.js. It supplies routing, middleware and request handling for the REST APIs.
- **PostgreSQL** is a reliable open-source relational database that supports constraints, indexes, transactions and powerful SQL queries, all of which are needed for the bin, reading and collection data.
- **Postman** is used to send requests to the APIs, check the responses and organize the requests into collections so that tests can be repeated.
- **Docker Desktop** packages the API, the simulator and the PostgreSQL database into containers so that the system behaves identically on every machine.

CHAPTER - 3 TECHNOLOGY STACK

3.1 Front-End Design

The Smart Waste Management System is designed as an API-first backend system. Its main purpose is to monitor bins, manage data and control the collection workflow, and all of this logic lives in the backend and the database. For this reason the current version does not include a separate graphical web interface. Instead, the REST APIs are the interface of the system, and **Postman** acts as the client that sends requests and displays the JSON responses.

This choice keeps the project focused on the parts that matter most for a database-driven system: a correct data model, reliable business rules and well-tested APIs. Because every function is exposed as an HTTP endpoint that returns JSON, any front end can be added later without changing the backend. A web dashboard for administrators and a mobile application for collectors are planned as future enhancements (see Chapter 11), and they would simply call the same APIs.

The role of a front end in the present version is therefore played by:

- **Postman collections**, which hold the requests for registration, login, bin management, readings, collection requests and reports.
- **JSON responses**, which present data in a structured, readable format for each request.
- **The simulator console**, which prints the readings it generates so that the flow of data can be observed while the system runs.

3.2 Back-End Design

The backend is developed with **Node.js** and **Express.js**. Node.js uses a non-blocking, event-driven model, which suits a system in which many readings and requests may arrive at the same time. Express.js adds routing and middleware on top of Node.js and keeps the code organized.

The backend follows a modular structure that mirrors the six modules of the system. Each module has its own routes, controller functions and database queries:

- **Routes** map URLs and HTTP methods (GET, POST, PUT, DELETE) to controller functions.
- **Controllers** hold the business logic, such as comparing a reading with the threshold or changing the status of a collection request.
- **Middleware** performs common tasks before the controller runs: parsing JSON bodies, checking the login token, verifying the user role and handling errors.

- **Database access layer** sends parameterized SQL queries to PostgreSQL through the pg driver, which also protects against SQL injection.

Node.js is used a second time for **data simulation**. A separate script generates waste-level readings for all registered bins at regular intervals and sends them to the monitoring API, exactly as a physical sensor network would do.

3.3 Database Design

PostgreSQL is the heart of the system. It was chosen for the following reasons:

- The data is highly structured and interrelated (a bin has many readings, a bin has many collection requests, a request belongs to a collector and a vehicle), which fits a relational model naturally.
- Primary keys, foreign keys, unique and check constraints enforce data integrity automatically.
- Transactions guarantee that related changes, such as completing a request and resetting the bin status, succeed or fail together.
- Indexes and aggregate functions (COUNT, AVG, MAX, GROUP BY) make reports such as bin-wise and location-wise analysis simple and fast.
- It is open source, widely supported, and has an official Docker image that makes it easy to run in a container.

The schema contains seven tables: `roles`, `users`, `bins`, `waste_levels`, `collectors`, `vehicles` and `collection_requests`. The complete design is explained in Chapter 5.

3.4 Version Control

The project source code, SQL scripts, Docker configuration and Postman collections are maintained under **Git** version control. Version control gives the team a history of every change, makes it possible to return to an earlier working state, and allows the three team members to work on different modules at the same time without overwriting each other's work.

Repository link: <https://github.com/iliharshika/Smart-Waste-Management-System-DBSE-DBD>

3.5 APIs or External Services

The system does not depend on any paid or third-party external service. Its data sources and interfaces are:

Component	Type	Description
Backend REST APIs	Internal	Express.js endpoints for authentication, bins, readings, collections, collectors, vehicles and reports
Data simulator	Internal (sensor substitute)	Node.js program that generates realistic waste-level readings and posts them to the monitoring API
PostgreSQL	Database service	Runs as a Docker container and is accessed by the backend through the <code>pg</code> driver
Postman	Testing tool	Sends test requests and verifies API responses

In a future version, real IoT sensors would replace the simulator and send readings to the same API.

CHAPTER - 4 SYSTEM ARCHITECTURE

4.1 High-Level Architecture Diagram

The system is organized into four layers, shown in Fig 4.1. Clients send requests to the backend, the backend applies the business rules and talks to the database, and a separate simulation layer feeds waste-level readings into the system. The backend, the simulator and the database run as containers under Docker Desktop.

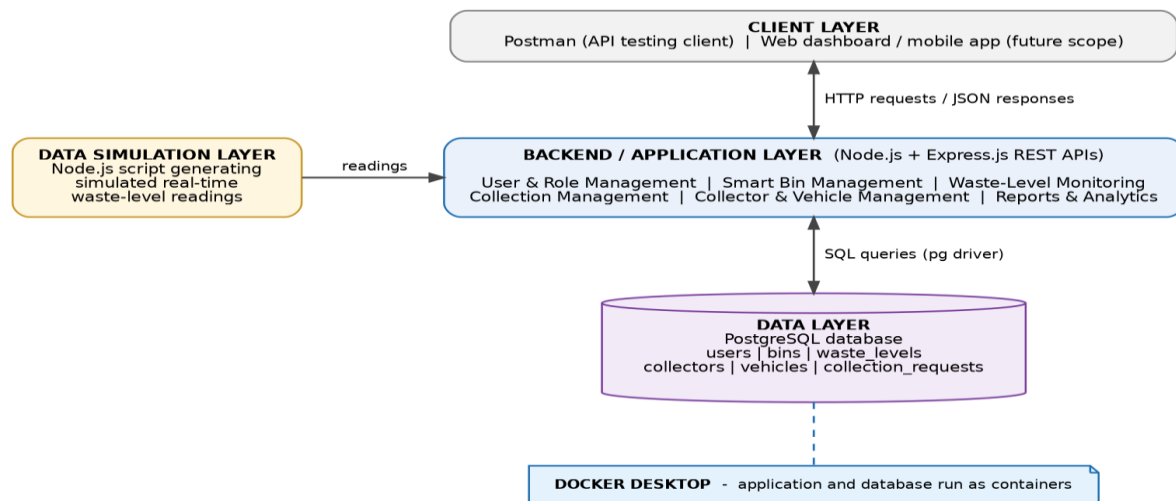


Fig 4.1 High-Level Architecture of the Smart Waste Management System

4.2 Description of Each Layer (Frontend, Backend, Database)

Client layer

This layer represents whatever sends requests to the system. In the present version it is Postman, which issues HTTP requests and displays the JSON responses. Because the backend exposes a clean REST interface, a web dashboard or mobile application can be added to this layer later without any change to the layers below.

Data simulation layer

This layer stands in for the physical sensors. A Node.js program reads the list of registered bins and, at regular intervals, generates a fill-level percentage for each one. The generation logic changes each bin's level gradually and realistically, so that levels rise over time, differ from bin to bin, and fall back after a collection. Every generated reading is posted to the waste-level monitoring API.

Backend / application layer

This layer is built with Node.js and Express.js and contains the six modules: User & Role Management, Smart Bin Management, Waste-Level Monitoring, Collection Management, Collector & Vehicle Management, and Reports & Analytics. It validates input, authenticates

users, enforces role-based access, applies the threshold logic, manages the lifecycle of collection requests and returns results as JSON. It is the only layer that talks directly to the database.

Data layer

PostgreSQL stores all persistent data: users and roles, bins, every waste-level reading, collectors, vehicles and collection requests. Foreign keys connect these tables so that, for example, every reading belongs to a valid bin and every request refers to a valid bin, collector and vehicle. Keeping all data in one database gives a single source of truth and makes complete historical reports possible.

Data flow

A reading flows through the system as follows: the simulator generates a reading, sends it to the monitoring API, the backend stores it in `waste_levels` and compares it with the bin's threshold, and, if the threshold is crossed, a collection request is created in `collection_requests`. A supervisor assigns the request to a collector and a vehicle, the collector updates its status until completion, and the stored records are later summarized by the reporting APIs.

4.3 Deployment Architecture (Cloud, Local, CI/CD Pipelines)

The system is deployed **locally using Docker Desktop**. Three containers are defined and started together with Docker Compose:

Container	Image / Base	Role
db	Official PostgreSQL image	Stores all data; uses a named volume so that data survives container restarts; initialized with the schema SQL script
api	Node.js image built from the project Dockerfile	Runs the Express.js backend and exposes the REST API on a mapped port
simulator	Node.js image (same code base)	Runs the data-simulation script that posts readings to the API

Containerization means the environment is described in files and not set up by hand, so the complete system can be started on any machine with a single command and behaves identically everywhere. Configuration values such as database host, user, password and port are supplied through environment variables.

Cloud and CI/CD. Deployment to a cloud platform and an automated CI/CD pipeline are not part of the current version. Because the system is already containerized, moving it to a cloud

service later would only require pushing the images to a registry and running them on the chosen platform. This is listed under Future Scope in Chapter 11.

CHAPTER - 5 DESIGN

5.1 System Workflow Design

Fig 5.1 shows the central workflow of the system. Everything starts with a waste-level reading. The reading is stored and compared with the threshold of the bin. If the level is below the threshold, monitoring simply continues. If the level is at or above the threshold, a collection request is created, assigned to a collector and a vehicle, tracked through its statuses and finally completed. The result is kept as history for reports.

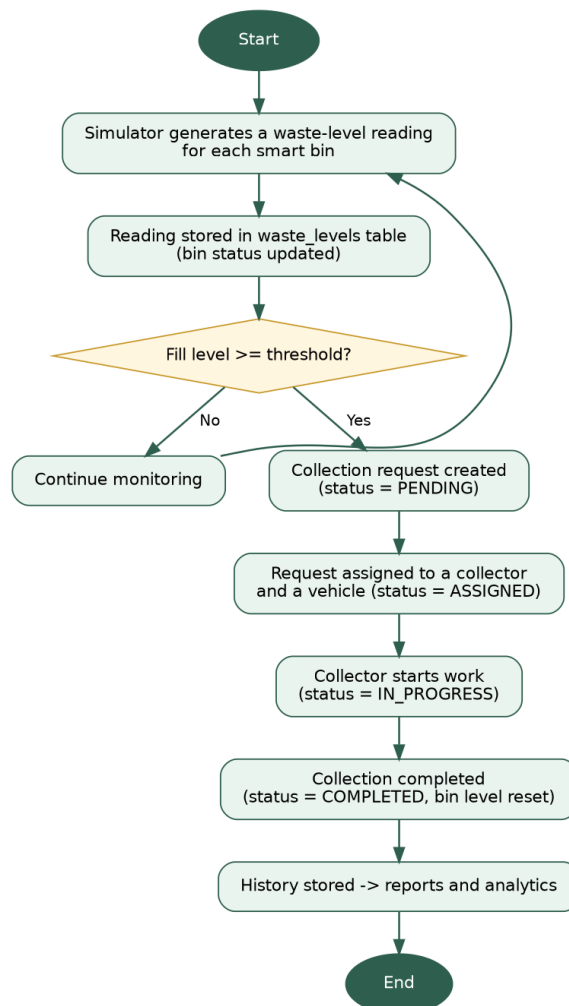


Fig 5.1 Workflow: from Waste-Level Reading to Collection Records

The status of a collection request moves through four states: **PENDING** (created, waiting for assignment), **ASSIGNED** (a collector and vehicle have been allocated), **IN_PROGRESS** (the collector has started) and **COMPLETED** (the bin has been emptied). A request cannot skip a state, which keeps the records reliable.

5.2 ER Diagram / Database Schema

The database design is shown in the entity-relationship diagram in Fig 5.2. It consists of seven related tables.

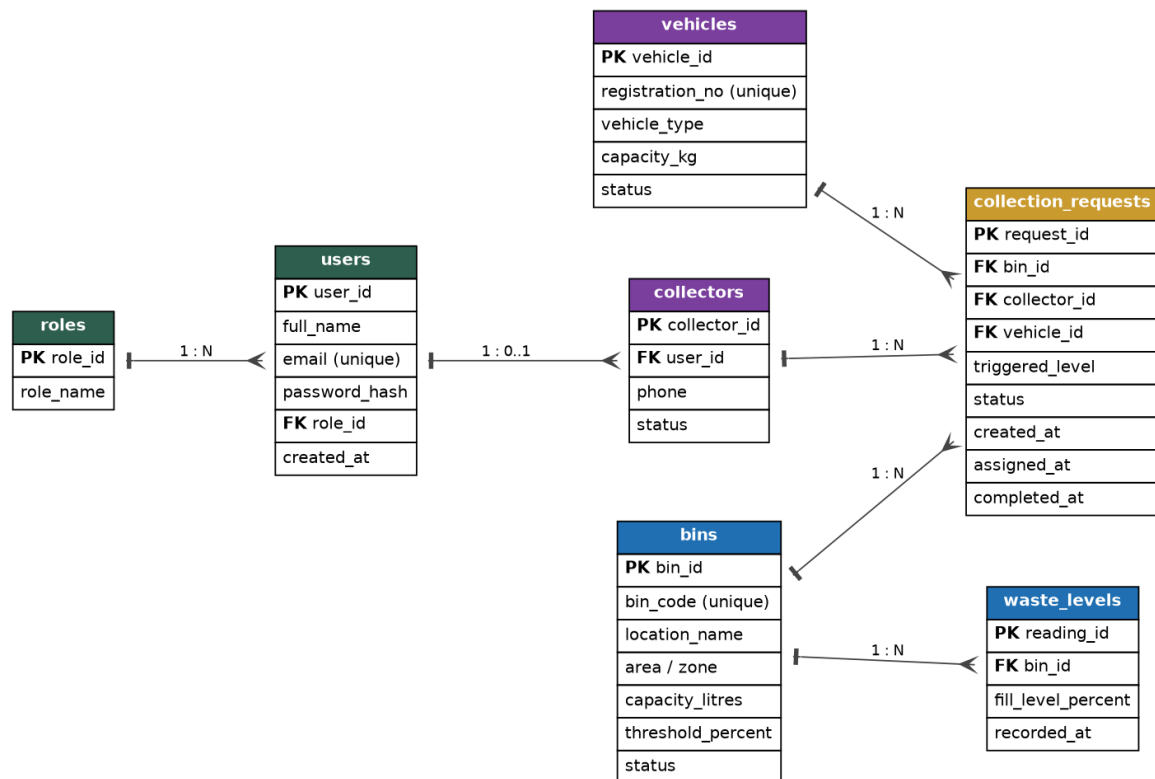


Fig 5.2 Entity-Relationship Diagram (Database Schema)

The relationships are:

- One **role** is assigned to many **users** (1 : N).
- A **user** may also be registered as a **collector** (1 : 0..1).
- One **bin** has many **waste-level readings** (1 : N).
- One **bin** can have many **collection requests** over time (1 : N).
- One **collector** and one **vehicle** can each be assigned to many **collection requests** (1 : N).

Table descriptions

Table	Key columns	Purpose
roles	role_id (PK), role_name	Defines user roles such as Admin, Supervisor and Collector for access control
users	user_id (PK), email (unique), password_hash, role_id (FK)	Stores registered users and their credentials; passwords are stored only as hashes

Table	Key columns	Purpose
bins	bin_id (PK), bin_code (unique), location_name, area, capacity_litres, threshold_percent, status	Stores each smart bin with its location, capacity and the threshold used for detection
waste_levels	reading_id (PK), bin_id (FK), fill_level_percent, recorded_at	Stores every simulated reading; this table is the waste-level history
collectors	collector_id (PK), user_id (FK), phone, status	Stores collector details and availability
vehicles	vehicle_id (PK), registration_no (unique), vehicle_type, capacity_kg, status	Stores collection vehicle information and availability
collection_requests	request_id (PK), bin_id, collector_id, vehicle_id (FKs), triggered_level, status, created_at, assigned_at, completed_at	Tracks each collection from creation to completion; also serves as the collection history

Design decisions

- **Normalization:** the schema is in third normal form; for example, location and capacity are stored once in `bins` and not repeated in every reading.
- **Integrity:** primary keys, foreign keys, unique constraints and CHECK constraints (such as `fill_level_percent BETWEEN 0 AND 100` and a fixed list of valid statuses) are enforced by the database itself.
- **Performance:** an index on `waste_levels (bin_id, recorded_at)` speeds up retrieval of the latest reading and the history of a bin, and an index on `collection_requests (status)` speeds up listing of pending work.
- **History:** readings and requests are never overwritten; new rows are added, so complete history is always available for reports.

CHAPTER - 6 IMPLEMENTATION

6.1 Module-Wise Implementation

The system is implemented as six modules. Each module has its own routes, controllers and database queries, but all of them share the same PostgreSQL database, as shown in Fig 6.1. This keeps the code organized while ensuring that data stays consistent across the system.

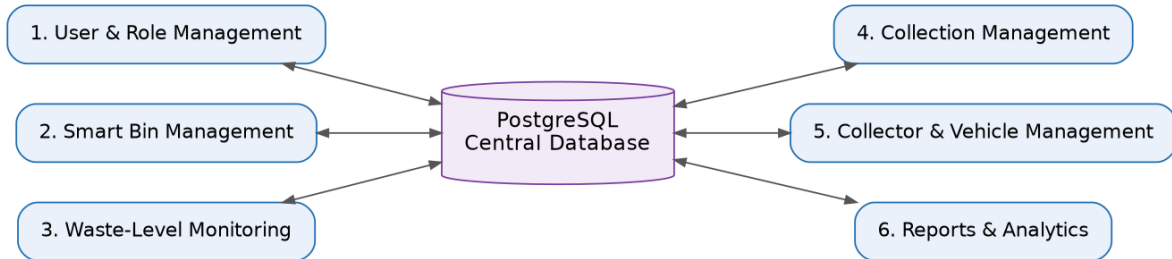


Fig 6.1 Six Core Modules Sharing One Central Database

Module 1: User & Role Management

This module handles registration and login, stores user details and roles, and controls access. A new user registers with a name, email and password; the password is hashed before storage. On login, the system verifies the credentials and issues a token that identifies the user and the role in later requests. Access control middleware then allows or blocks each endpoint according to the role. For example, only an administrator may add bins, and only an assigned collector may update the status of his or her request.

Module 2: Smart Bin Management

This module adds and manages bins. For each bin it stores a unique code, the location and area, the capacity, the threshold percentage at which the bin should be collected, and the current status (for example ACTIVE or UNDER_MAINTENANCE). Administrators can create, view, update and deactivate bins. Deactivated bins are ignored by the monitoring logic, but their history is preserved.

Module 3: Waste-Level Monitoring

This module receives and stores the simulated readings. When a reading arrives, the module validates that the bin exists and that the level is between 0 and 100 percent, inserts the reading into `waste_levels`, and updates the status of the bin. It then compares the level with the threshold of the bin. If the threshold is crossed and the bin has no open collection request, the module triggers the creation of a new request. This check for an open request prevents duplicate requests for the same bin. The module also returns the latest level of every bin and the level history of a single bin.

Module 4: Collection Management

This module manages the lifecycle of collection requests. A request is created automatically by the monitoring module or manually by a supervisor. A supervisor assigns it to an available collector and vehicle; the collector marks it IN_PROGRESS when work begins and COMPLETED when the bin is emptied. At each step the module records a timestamp and ensures that statuses change only in the allowed order (PENDING, ASSIGNED, IN_PROGRESS, COMPLETED). When a request is completed, the module marks the collector and vehicle as available again and records the emptied state of the bin.

Module 5: Collector & Vehicle Management

This module stores collector details (name, phone, availability) and vehicle information (registration number, type, capacity, availability). It also provides the lists of available collectors and vehicles that the supervisor uses when assigning a request, and it keeps the assignment and activity records, so that the work done by each collector and vehicle can be reviewed later.

Module 6: Reports & Analytics

This module produces summaries from the stored data using SQL aggregate queries. It provides waste-level reports, collection statistics (number of requests by status, average time from creation to completion), and bin-wise and location-wise analysis, such as which bins reach the threshold most often and which areas generate the most collection requests. These reports are the practical benefit of keeping organized historical data.

6.2 Front-End Logic (Client Interaction and Data Simulation)

Since the current system has no graphical front end, the "front-end logic" consists of how clients interact with the APIs and how the simulator behaves as the data-producing client.

Client interaction

Every client request follows the same pattern: the client sends an HTTP request with a JSON body (and a token for protected endpoints), the backend validates it and returns a JSON response containing either the requested data or a clear error message with an appropriate HTTP status code (200, 201, 400, 401, 403, 404 or 500). Postman is used to send these requests, and the responses show exactly what a future dashboard would receive.

Data simulation logic

The simulator is a Node.js script that runs as its own container. Its logic is as follows:

1. Fetch the list of active bins from the API.

2. Keep the current fill level of every bin in memory. At the start, each bin receives a random initial level.
3. At every cycle (for example every few seconds), increase each bin's level by a small random amount, with different rates for different bins so that busy locations fill faster than quiet ones.
4. Send each new level to the waste-level monitoring API as a reading.
5. When the backend reports that a bin has been collected, reset that bin's level to a low value so that the cycle starts again.

This approach produces data that resembles real sensor output: gradual rise, differences between bins, occasional rapid filling and sudden drops after collection. Because the simulator talks to the API in the same way a real sensor would, replacing it with physical sensors later requires no change to the backend.

6.3 API Integration

All functions of the system are exposed through REST APIs. The table below lists the main endpoints of each module. (Endpoint names follow REST conventions; protected endpoints require a valid token in the Authorization header.)

Module	Method	Endpoint	Description
Users	POST	/api/auth/register	Register a new user
Users	POST	/api/auth/login	Log in and receive a token
Users	GET	/api/users	List users (admin only)
Bins	POST	/api/bins	Add a new bin (admin)
Bins	GET	/api/bins	List all bins with latest status
Bins	PUT	/api/bins/:id	Update bin details or threshold
Monitoring	POST	/api/waste-levels	Store a reading (used by the simulator) and run threshold check
Monitoring	GET	/api/waste-levels/latest	Latest level of every bin
Monitoring	GET	/api/waste-levels/bin/:binId	Level history of one bin
Collection	GET	/api/collections?status=	List requests, optionally filtered by status
Collection	PUT	/api/collections/:id/assign	Assign a collector and vehicle
Collection	PUT	/api/collections/:id/status	Update request status
Collectors	POST / GET	/api/collectors	Add or list collectors
Vehicles	POST / GET	/api/vehicles	Add or list vehicles

Module	Method	Endpoint	Description
Reports	GET	/api/reports/waste-levels	Waste-level report
Reports	GET	/api/reports/collections	Collection statistics
Reports	GET	/api/reports/bin-wise	Bin-wise analysis
Reports	GET	/api/reports/location-wise	Location-wise analysis

Within the backend, controllers talk to PostgreSQL using the `pg` driver with parameterized queries. Data that must change together, such as completing a request and freeing the collector and vehicle, is executed inside a database transaction so that either all the changes are saved or none of them is.

6.4 Authentication and Authorization

Authentication (who is the user?) is performed at login. The password supplied by the user is compared with the stored password hash; passwords are never stored or returned in plain text. After a successful login, the backend issues a signed token that carries the user identifier and role. The client sends this token with every later request.

Authorization (what may this user do?) is enforced by middleware that reads the token, identifies the role and checks it against the permissions of the requested endpoint:

Role	Permissions
Admin	Manage users, bins, collectors and vehicles; view all reports
Supervisor	View bins and readings; create and assign collection requests; view reports
Collector	View assigned requests; update the status of own requests

A request without a valid token receives HTTP 401 (Unauthorized), and a request from a user whose role is not permitted receives HTTP 403 (Forbidden). This keeps the data protected while allowing every user to do the work that belongs to the role.

CHAPTER - 7 FEATURES

7.1 List of Main Features

- Registration, login and role-based access control.
- Smart bin registration with location, capacity and threshold.
- Simulated real-time waste-level monitoring.
- Automatic threshold detection and creation of collection requests.
- Assignment of requests to collectors and vehicles.
- Status tracking of requests from PENDING to COMPLETED.
- Collector and vehicle management with availability tracking.
- Complete history of waste levels and collections.
- Reports and analytics: waste-level reports, collection statistics, bin-wise and location-wise analysis.
- Containerized deployment with Docker for consistent setup.

7.2 How Each Feature Works

Registration, login and access control

User perspective: a person registers with name, email and password, logs in and receives a token; what the person can do afterwards depends on the assigned role. **Technical description:** the password is hashed before being stored in `users`; at login the hash is verified and a signed token is returned; middleware checks the token and role on every protected endpoint.

Smart bin registration

User perspective: an administrator adds a bin by entering its code, location, area, capacity and threshold. **Technical description:** a POST request inserts a row into `bins`; unique and CHECK constraints reject duplicate codes and invalid values; the bin then becomes visible to the monitoring module and to the simulator.

Simulated real-time monitoring

User perspective: users can view the current fill level of every bin and see how it changes over time. **Technical description:** the simulator posts a reading for each bin at regular intervals; the monitoring API stores it in `waste_levels` and the latest-level query returns the most recent row for each bin.

Threshold detection and automatic request creation

User perspective: when a bin becomes nearly full, a collection request appears automatically without anyone having to notice it. **Technical description:** after storing a reading, the backend compares `fill_level_percent` with the bin's `threshold_percent`. If the level is greater than or equal to the threshold and no PENDING, ASSIGNED or IN_PROGRESS request exists for that bin, a new PENDING request is inserted with the triggering level and timestamp.

Assignment and tracking of collection requests

User perspective: a supervisor sees pending requests, picks an available collector and vehicle, and the collector then updates progress. **Technical description:** the assign endpoint verifies that the request is PENDING and that the selected collector and vehicle are available, writes their identifiers and the assignment time to the request, and marks them as busy, all inside one transaction. The status endpoint allows only the next valid state.

Collector and vehicle management

User perspective: administrators maintain the list of staff and vehicles and always know who is available. **Technical description:** `collectors` and `vehicles` have a status column that is set to BUSY on assignment and to AVAILABLE when a request is completed.

History and reports

User perspective: managers can see how much waste each bin and location produces, how many collections were done and how long they took. **Technical description:** report endpoints run SQL aggregate queries (COUNT, AVG, MAX with GROUP BY and joins between `bins`, `waste_levels` and `collection_requests`) and return the results as JSON. Because no rows are overwritten, the history is complete.

Containerized deployment

User perspective: the whole system starts with a single command and works the same on any computer. **Technical description:** Docker Compose starts the PostgreSQL, API and simulator containers with the correct network and environment settings; a named volume keeps the database data.

CHAPTER - 8 TESTING

8.1 Postman Testing (Frontend/Backend)

Since the backend REST APIs are the interface of the system, **Postman** is the main tool for testing it. Postman plays the role of the front end: it sends requests exactly as a real client would and shows the status code, response time and JSON body of each response.

The tests were organized as a Postman collection with one folder per module (Users, Bins, Monitoring, Collection, Collectors & Vehicles, Reports). Environment variables hold the base URL and the token, so that the same requests can be reused without editing. Every endpoint was tested in three ways:

- **Positive tests:** valid input that should succeed, checking the expected status code (200 or 201) and response fields.
- **Negative tests:** invalid input, such as a missing field, an unknown bin, a level above 100, or a duplicate email, checking that a proper error (400, 404 or 409) is returned and no wrong data is stored.
- **Security tests:** requests without a token (expected 401) and requests with the wrong role (expected 403).

Postman's Tests tab was used to write simple assertions, for example to check that the status code is 201 after creating a bin, and that the returned JSON contains the new `bin_id`. After each test the database tables were also checked to confirm that the stored data matched the response.

8.2 Integration Testing

Integration testing checks that the modules work correctly together, not only in isolation. The most important integration scenario is the full workflow from reading to completed collection:

1. Register an admin and log in; add several bins with different thresholds.
2. Start the simulator and confirm that readings appear in `waste_levels` for every bin.
3. Let a bin cross its threshold and confirm that exactly one PENDING collection request is created for it.
4. Register a collector and a vehicle; assign the request and confirm that both become BUSY.
5. Update the request to IN_PROGRESS and then to COMPLETED; confirm that the collector and vehicle become AVAILABLE and that the bin level resets.

6. Call the report endpoints and confirm that the completed request appears in the statistics.

A second integration check verifies the containers: the API must be able to reach the database container by its service name, the schema must be created automatically on first start, and the data must still be present after the containers are stopped and started again.

8.3 Tools Used for Testing

Tool	Use
Postman	Sending test requests, writing assertions and organizing test collections
Docker Desktop	Running the API, simulator and database in an environment identical to deployment
Visual Studio Code	Viewing logs, debugging the backend and running the simulator
PostgreSQL client (psql or a GUI tool)	Checking that the stored rows match the API responses

8.4 Test Cases and Results

The table below lists the main test cases. Each case gives the test input and the expected result. The last column records the outcome after execution.

ID	Module	Test case (input)	Expected result
TC-01	Users	Register with valid name, email, password	201 Created; user saved with hashed password
TC-02	Users	Register with an already used email	409 / 400 error; no duplicate row
TC-03	Users	Login with correct credentials	200 OK; token returned
TC-04	Users	Login with wrong password	401 Unauthorized
TC-05	Bins	Add a bin with valid data (admin token)	201 Created; bin visible in list
TC-06	Bins	Add a bin with a duplicate bin_code	Error; no duplicate row
TC-07	Bins	Add a bin without a token	401 Unauthorized
TC-08	Monitoring	Post a reading of 45 % for an existing bin	201 Created; no collection request created
TC-09	Monitoring	Post a reading above the threshold	201 Created; one PENDING request created
TC-10	Monitoring	Post a second reading above the threshold for the same bin	No duplicate request created

ID	Module	Test case (input)	Expected result
TC-11	Monitoring	Post a reading of 120 %	400 Bad Request; reading rejected
TC-12	Monitoring	Post a reading for a non-existent bin	404 Not Found
TC-13	Monitoring	Get level history of a bin	Readings returned in time order
TC-14	Collection	Assign a PENDING request to an available collector and vehicle	Status ASSIGNED; collector and vehicle BUSY
TC-15	Collection	Assign a request to a BUSY collector	Error; request unchanged
TC-16	Collection	Set status IN_PROGRESS then COMPLETED	Statuses updated with timestamps
TC-17	Collection	Try to set COMPLETED directly from PENDING	Error; invalid status change
TC-18	Collection	Collector updates another collector's request	403 Forbidden
TC-19	Reports	Get bin-wise and location-wise reports	Correct counts and averages returned
TC-20	Deployment	Restart all containers	Data still present; API reconnects to database

The Status column is to be filled with **Pass** or **Fail** (with remarks) after the test cases are executed and the Postman results are captured. Any failed case should be corrected and re-tested before the final submission.

CHAPTER - 9 DEPLOYMENT

9.1 Steps to Deploy

The system is deployed on a local machine using Docker Desktop. Because all components are containerized, deployment consists of preparing the configuration and starting the containers.

1. Install Docker Desktop and make sure it is running.
2. Copy the project folder to the machine (clone the repository or extract the ZIP file).
3. Create the `.env` file in the project root with the database and application settings (see Section 9.2).
4. Open a terminal in the project folder and run `docker compose up --build`. Docker builds the API image and starts three containers: `db`, `api` and `simulator`.
5. On its first start, the PostgreSQL container runs the schema script automatically and creates all tables.
6. Check that the containers are running (`docker ps`) and that the API responds at `http://localhost:3000`.
7. Import the Postman collection and run the login request to begin using the system.
8. To stop the system, run `docker compose down`. The database data is kept in a Docker volume and remains available at the next start.

The deployment is described by two files. The **Dockerfile** builds the Node.js image used by the API and the simulator:

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install --omit=dev
COPY . .
EXPOSE 3000
CMD ["node", "src/server.js"]
```

The **docker-compose.yml** file defines the three services and the volume that stores database data:

```
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_USER: ${DB_USER}
      POSTGRES_PASSWORD: ${DB_PASSWORD}
      POSTGRES_DB: ${DB_NAME}
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./sql/schema.sql:/docker-entrypoint-initdb.d/schema.sql
    ports: ["5432:5432"]
  api:
    build: .
    env_file: .env
    depends_on: [db]
```

```

    ports: ["3000:3000"]
    simulator:
      build: .
      command: node simulator/index.js
      env_file: .env
      depends_on: [api]
    volumes:
      pgdata:

```

9.2 Environment Configuration

Settings that change between machines or contain secrets are kept out of the source code and supplied through environment variables in a `.env` file. The `.env` file is excluded from version control.

Variable	Example value	Purpose
DB_HOST	db	Hostname of the database container (the Compose service name)
DB_PORT	5432	PostgreSQL port
DB_USER	waste_user	Database user
DB_PASSWORD	(secret)	Database password
DB_NAME	waste_db	Database name
PORT	3000	Port on which the API listens
JWT_SECRET	(secret)	Key used to sign login tokens
API_URL	http://api:3000	Address used by the simulator to reach the API
SIM_INTERVAL_MS	5000	Time between simulator cycles

Inside the Docker network the containers find each other by service name, which is why the database host is `db` and not `localhost`.

9.3 Hosting of Frontend and Backend

Backend and database: the API, the simulator and PostgreSQL are hosted locally as Docker containers on the machine running Docker Desktop. The API is reachable at `http://localhost:3000` and the database at port 5432 (used only by the backend and by database tools).

Frontend: there is no separate front-end application in the current version; Postman is used as the client. A future dashboard can be hosted separately and pointed to the API address.

Moving to the cloud: the same Docker images can run on a cloud virtual machine or container service, with a managed PostgreSQL database; only the environment variables change. This is part of the future scope.

CHAPTER - 10 CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

Several technical and design challenges had to be addressed while developing the system:

- **Realistic data without sensors:** random numbers alone produce unrealistic readings that jump up and down. The simulator needed to behave like real bins, with gradual filling, different speeds for different bins and a drop after collection.
- **Duplicate collection requests:** a bin that stays above the threshold keeps producing readings, and each reading could create another request for the same bin.
- **Choosing the threshold:** a low threshold causes frequent unnecessary collection, while a high one risks overflow.
- **Keeping related data consistent:** assigning or completing a request changes several tables (request, collector, vehicle), and a failure half-way would leave inconsistent records.
- **Container networking:** the API container could not reach the database using `localhost`, and it sometimes started before the database was ready.
- **Growth of the readings table:** continuous readings from many bins add rows quickly, which can slow down history and report queries.
- **Secure handling of credentials:** passwords and secret keys had to be kept out of the source code.

10.2 Solutions Applied

Problem	Solution
Unrealistic readings	The simulator keeps the current level of every bin and increases it by a small random amount with a bin-specific rate; levels reset after collection
Duplicate requests	A new request is inserted only if no PENDING, ASSIGNED or IN_PROGRESS request already exists for that bin
Threshold choice	The threshold is stored per bin in the <code>bins</code> table, so it can be tuned for each location without changing code
Inconsistent data	Multi-table changes are executed inside database transactions; status changes follow a fixed order
Container networking	Containers use the Compose service name as the database host; the API retries the connection until the database is ready
Table growth	A composite index on <code>(bin_id, recorded_at)</code> was added, and queries always filter by bin or time range

Problem	Solution
Credentials	Secrets are stored in environment variables in a <code>.env</code> file that is not committed to version control; passwords are stored only as hashes

10.3 Current Limitations or Known Bugs

- **Simulated data:** readings are generated by a program and not by physical sensors, so the system shows the working of the concept but not field accuracy.
- **No vehicle tracking:** vehicle position is not captured, because GPS is not integrated.
- **No route optimization:** the system decides which bins need collection, but not the best order in which to visit them.
- **No graphical interface or mobile app:** the system is operated through APIs and Postman.
- **No automatic alerts:** staff are not notified when a bin reaches a critical level; requests must be viewed through the API.
- **No prediction:** the system reacts to the current level and does not estimate future waste generation.
- **Local deployment only:** the system runs on a single machine under Docker Desktop and is not yet accessible from multiple locations.

Each of these limitations is addressed by an item in the future scope described in Chapter 11.

CHAPTER - 11 FUTURE ENHANCEMENTS

11.1 Planned Features

No.	Feature	Description
1	IoT sensors	Replace the simulator with ultrasonic fill-level sensors on actual smart bins that send readings to the same monitoring API
2	GPS tracking	Track the live position of collection vehicles to improve accountability and estimate arrival times
3	Route optimization	Calculate efficient routes that visit only the bins requiring collection, reducing distance, fuel and time
4	Mobile app	Let collectors receive assigned requests and update their status directly from a phone
5	Real-time alerts	Notify supervisors and collectors by SMS, email or push notification when a bin reaches a critical level
6	Predictive analysis	Use historical readings to estimate future waste generation for each bin and location and plan collection in advance
7	Cloud deployment	Host the system in the cloud so that it can be accessed from multiple locations and scaled as needed
8	Smart-city integration	Extend the system to a city-wide scale and connect it with other smart-city services and dashboards

11.2 Possible Integrations or Optimizations

- **Web dashboard:** a browser-based dashboard with a map of bins coloured by fill level, and charts built from the report APIs.
- **Messaging protocols:** MQTT or HTTP gateways for receiving readings efficiently from many sensors.
- **Mapping services:** integration with a maps API for bin location display, distance calculation and route generation.
- **Time-series storage and partitioning:** partitioning the `waste_levels` table by time, or using a time-series extension of PostgreSQL, to keep queries fast as data grows.
- **Caching:** caching frequently requested results, such as the latest level of every bin, to reduce database load.
- **Automated testing and CI/CD:** running the Postman collection automatically on every commit and deploying the verified containers to the cloud.
- **Machine learning:** training a forecasting model on the stored history to predict when each bin will cross its threshold.

- **Segregation data:** recording waste type (wet, dry, recyclable) per bin to support recycling analysis.

CHAPTER - 12 CONCLUSION

12.1 Summary of the Project

This project presented the design and development of a **Smart Waste Management System**, a database-driven system that uses simulated real-time waste-level data to monitor bins and manage waste collection. The work began with the observation that fixed collection schedules and manual monitoring cause overflowing bins, unnecessary trips, wasted fuel and manpower, and poor records.

To address these problems, the system was divided into six modules: User & Role Management, Smart Bin Management, Waste-Level Monitoring, Collection Management, Collector & Vehicle Management, and Reports & Analytics. A Node.js program simulates the readings of sensors; a PostgreSQL database stores bins, readings, users, collectors, vehicles and collection requests; Node.js and Express.js REST APIs apply the business rules; Postman verifies the APIs; and Docker Desktop packages the application and database for consistent deployment.

The result is a system that follows a clear chain of steps: **Monitor, Detect, Assign, Track and Analyze**. Waste levels are monitored, bins that cross the threshold are detected, collection requests are assigned, their progress is tracked until completion, and the stored history is analyzed through reports.

12.2 What Was Achieved

- A normalized relational database schema with seven tables, keys and constraints that keep the data consistent.
- A simulation program that produces realistic waste-level readings for all bins.
- Automatic detection of bins that cross a predefined threshold, with protection against duplicate requests.
- A complete collection-request lifecycle (PENDING, ASSIGNED, IN_PROGRESS, COMPLETED) with timestamps.
- Role-based authentication and authorization for administrators, supervisors and collectors.
- Waste-level history and reports, including bin-wise and location-wise analysis.
- API testing with Postman and containerized deployment with Docker, which makes the system easy to reproduce.

Together, these results meet the six objectives set in Chapter 1. The system reduces dependence on fixed schedules and provides organized data that supports better collection planning and decision-making.

12.3 Skills Learned During Development

- **Database design:** entity-relationship modelling, normalization, constraints, indexing and writing SQL queries and transactions in PostgreSQL.
- **Backend development:** building modular REST APIs with Node.js and Express.js, including validation, middleware and error handling.
- **Security basics:** password hashing, token-based authentication and role-based access control.
- **Simulation:** designing a program that generates realistic data and behaves like a real sensor client.
- **API testing:** writing positive, negative and security test cases and assertions in Postman.
- **Containerization:** writing a Dockerfile and a Compose file, and managing configuration through environment variables.
- **Teamwork and documentation:** dividing work between team members, using version control, and documenting a software project in a structured way.

CHAPTER - 13 REFERENCES

Books

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database System Concepts (7th ed.). McGraw-Hill Education.

Elmasri, R., & Navathe, S. B. (2016). Fundamentals of Database Systems (7th ed.). Pearson.

Pressman, R. S., & Maxim, B. R. (2019). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill Education.

Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education.

Documentation and Tools

PostgreSQL Global Development Group. PostgreSQL Documentation. Retrieved from <https://www.postgresql.org/docs/>

OpenJS Foundation. Node.js Documentation. Retrieved from <https://nodejs.org/en/docs>

Express.js. Express - Node.js web application framework. Retrieved from <https://expressjs.com>

node-postgres. node-postgres (pg) Documentation. Retrieved from <https://node-postgres.com>

Postman. Postman Learning Center. Retrieved from <https://learning.postman.com>

Docker Inc. Docker Documentation. Retrieved from <https://docs.docker.com>

Microsoft. Visual Studio Code Documentation. Retrieved from <https://code.visualstudio.com/docs>

Waste Management References

Kaza, S., Yao, L., Bhada-Tata, P., & Van Woerden, F. (2018). What a Waste 2.0: A Global Snapshot of Solid Waste Management to 2050. World Bank.

Ministry of Housing and Urban Affairs, Government of India. Swachh Bharat Mission - Urban. Retrieved from <https://sbmurban.org>

Ministry of Environment, Forest and Climate Change, Government of India. (2016). Solid Waste Management Rules, 2016.

CHAPTER - 14 APPENDICES

Screenshots of the App

Web Application Screenshots

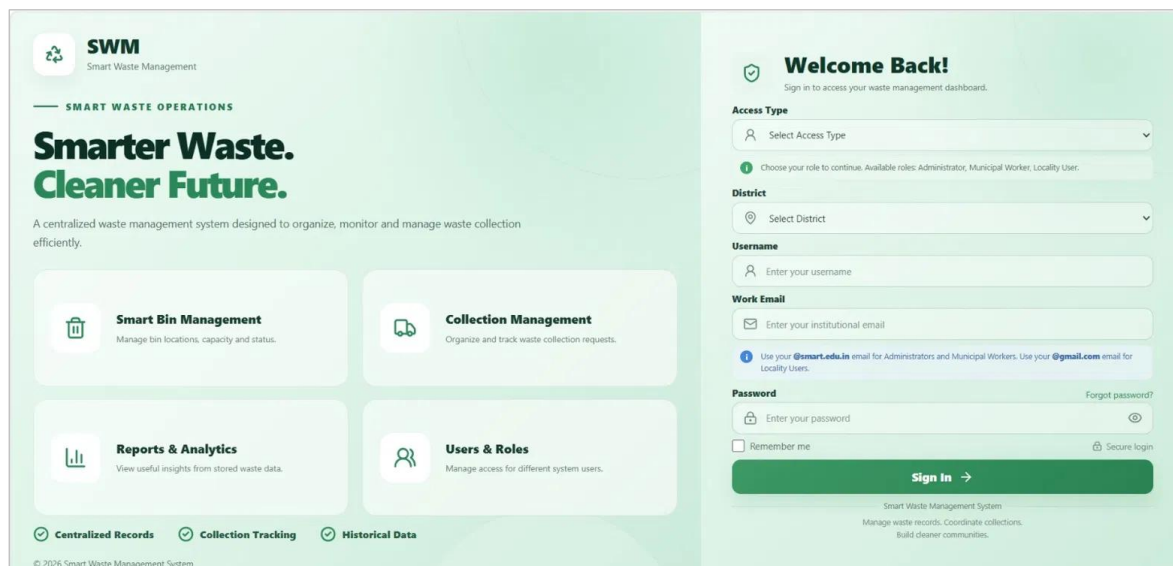


Figure A.8 - Login page

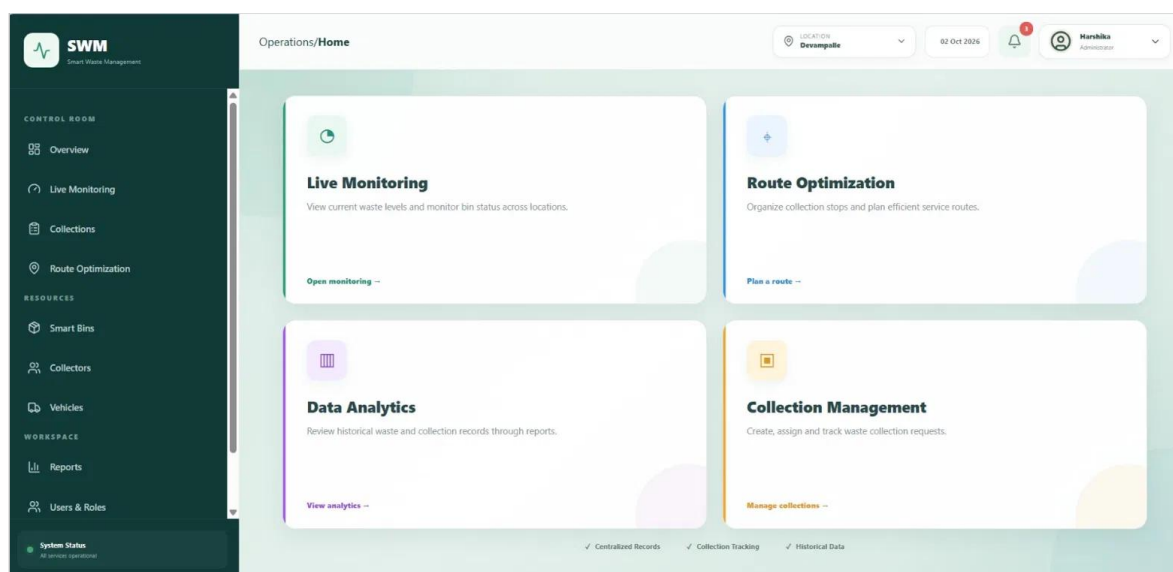


Figure A.9 - Dashboard home after sign-in

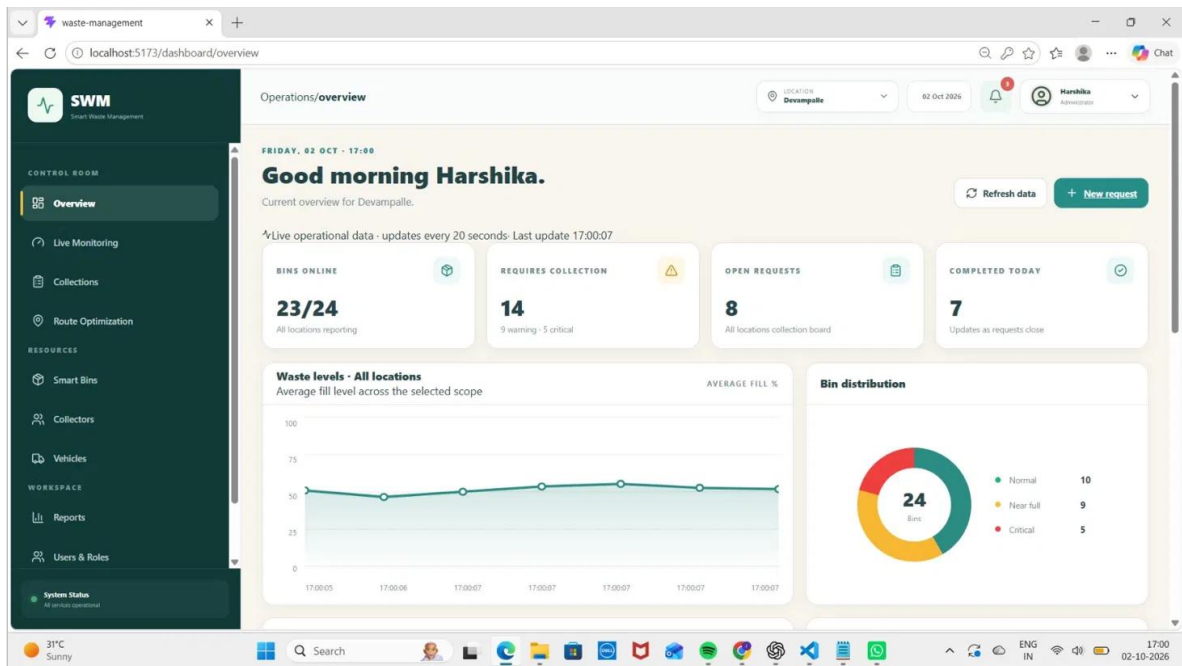


Figure A.10 - Overview

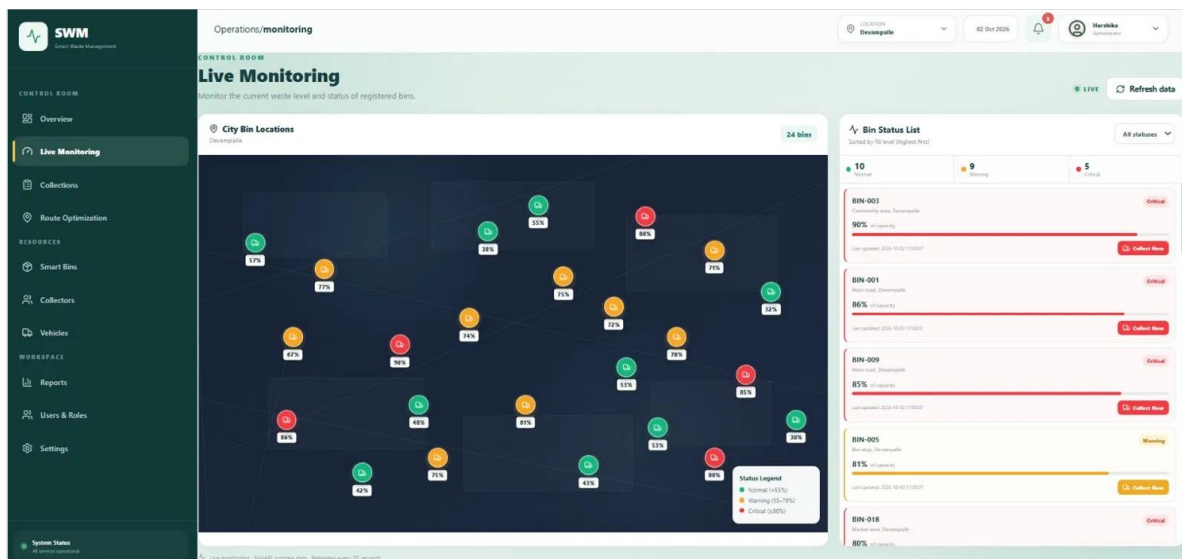


Figure A.11 - Live Monitoring

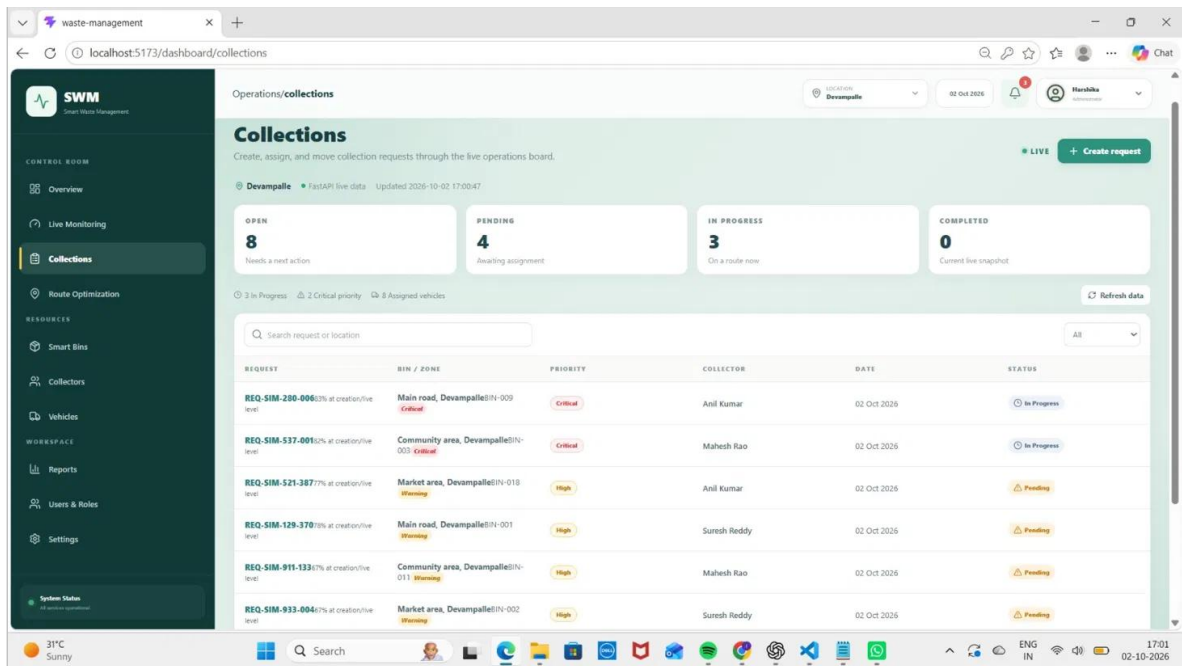


Figure A.12 - Collections

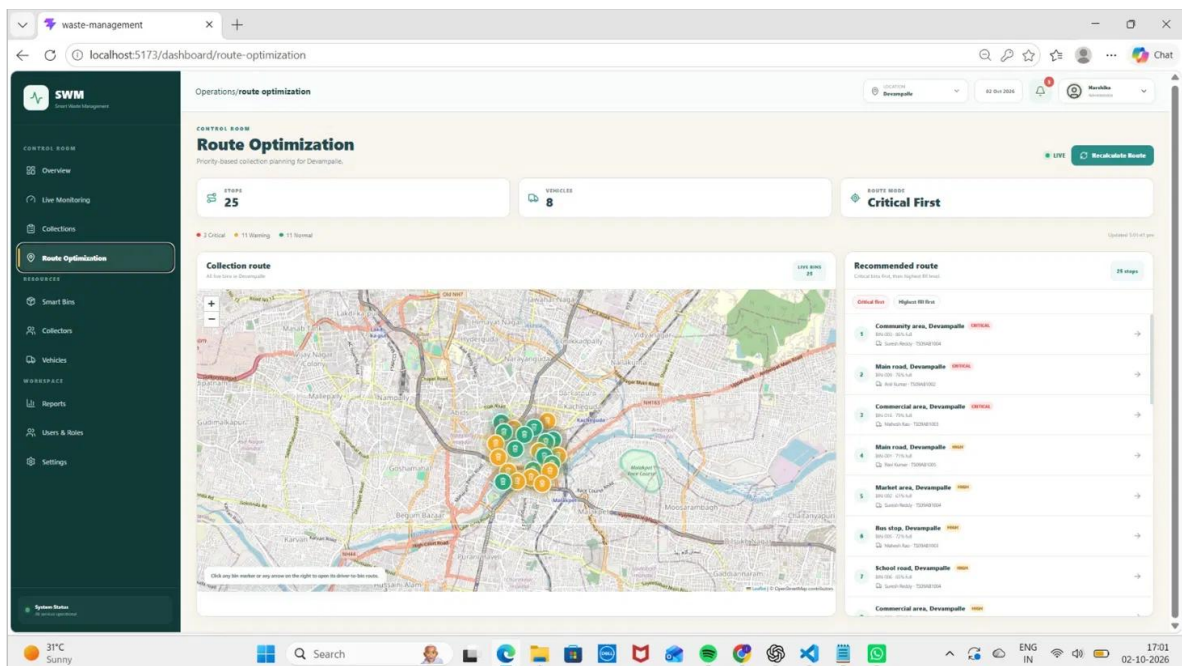


Figure A.13 - Route Optimization

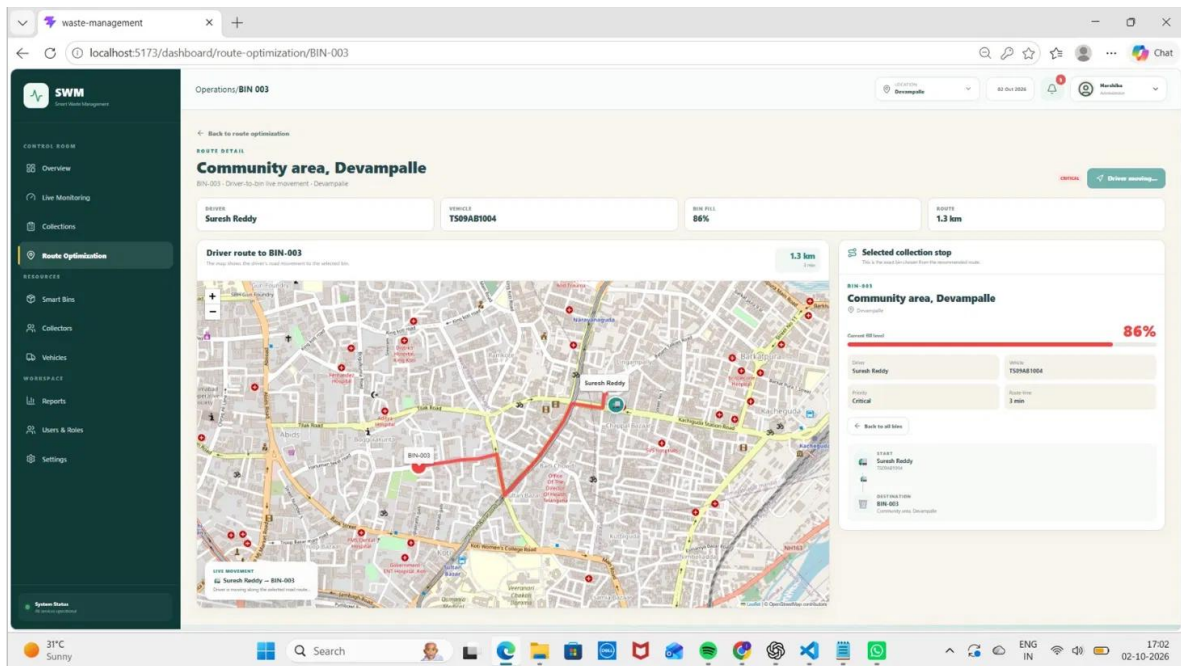


Figure A.14 - Route Optimization - driver route to a selected bin (BIN-003)

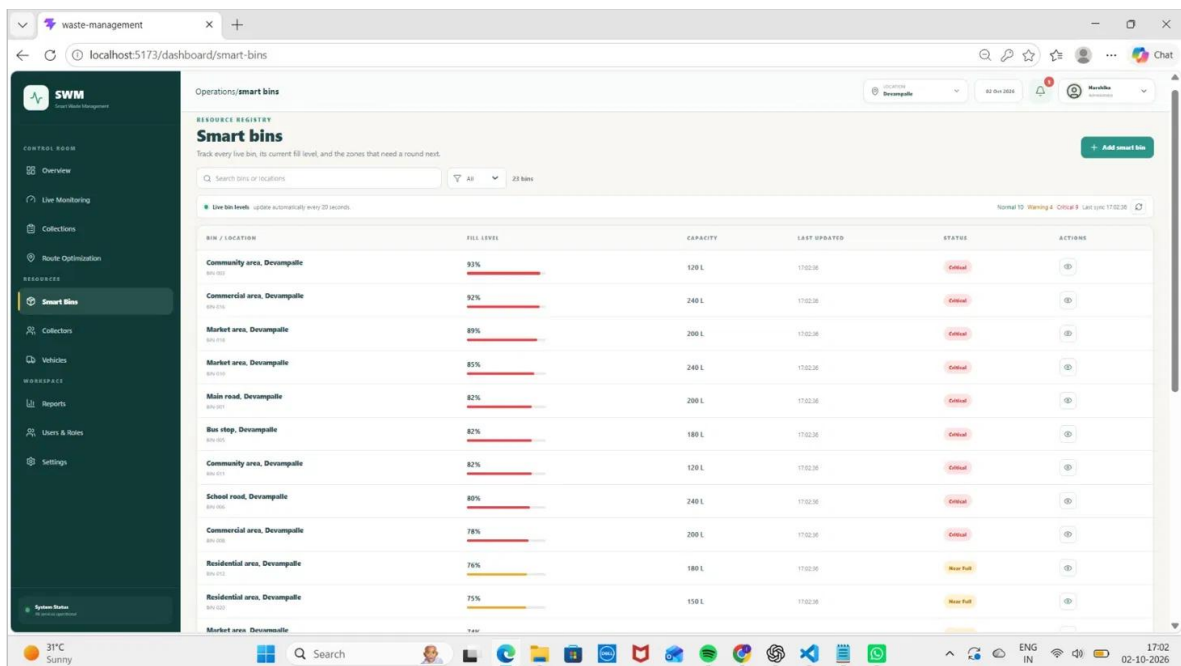


Figure A.15 - Smart Bins

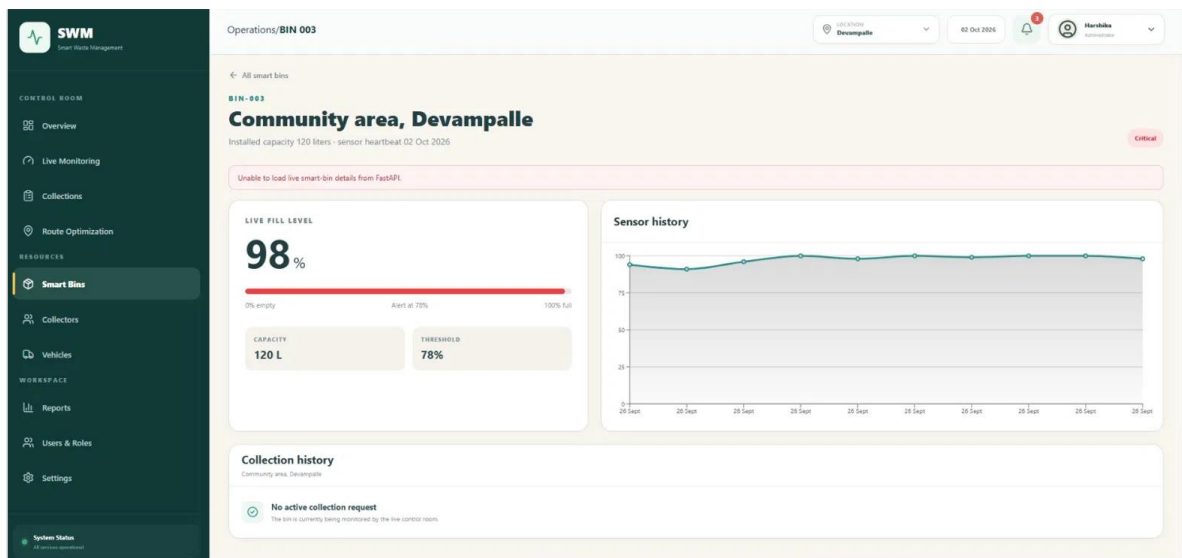


Figure A.16 - Smart Bin detail (BIN-003)

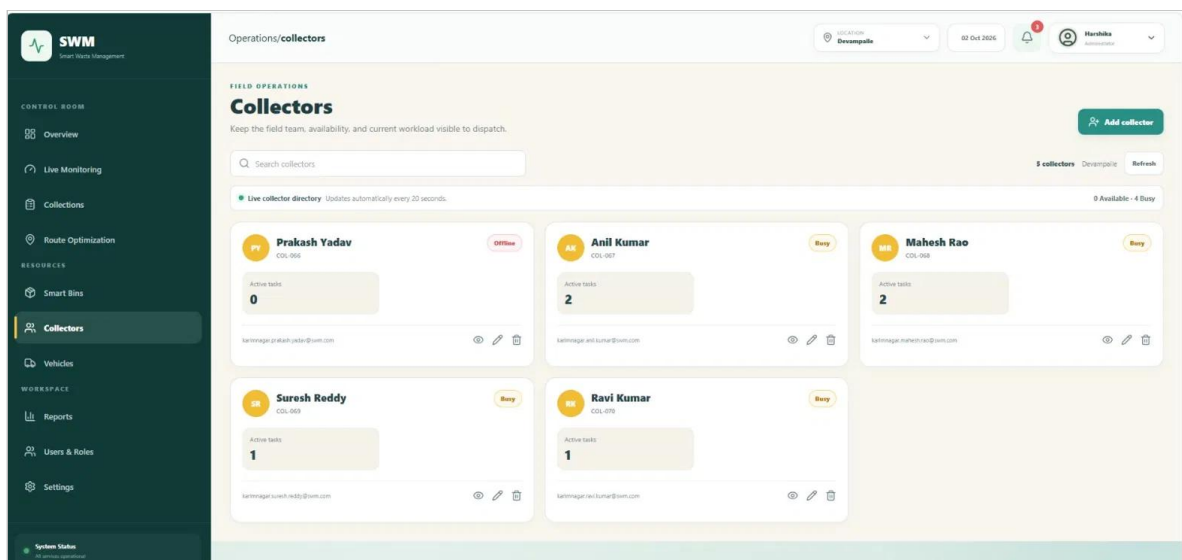


Figure A.17 - Collectors

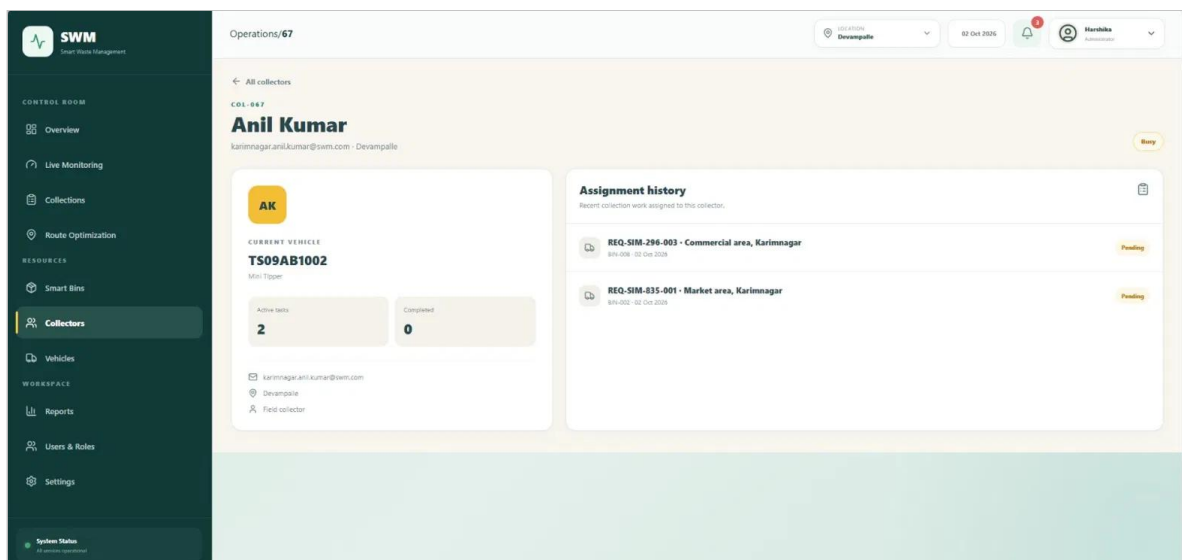


Figure A.18 - Collector detail (Anil Kumar)

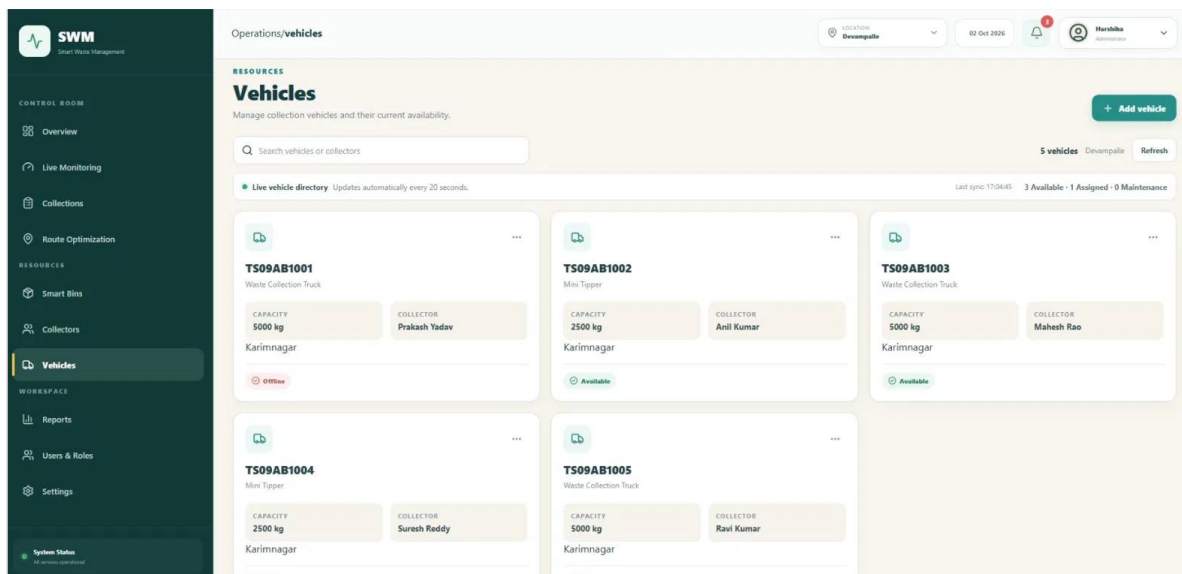


Figure A.19 - Vehicles

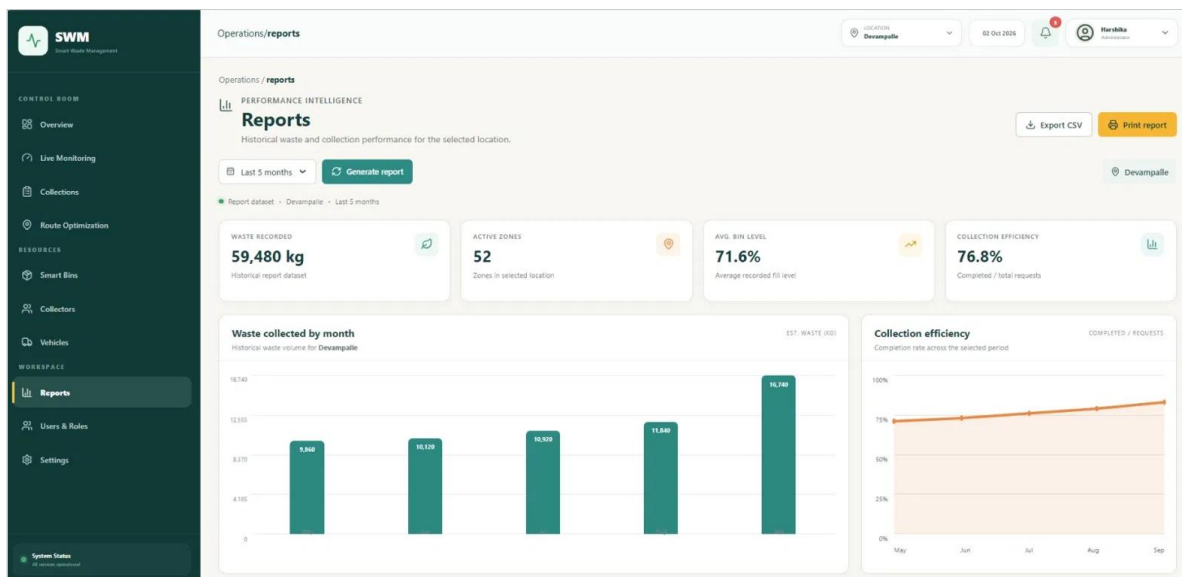


Figure A.20 - Reports

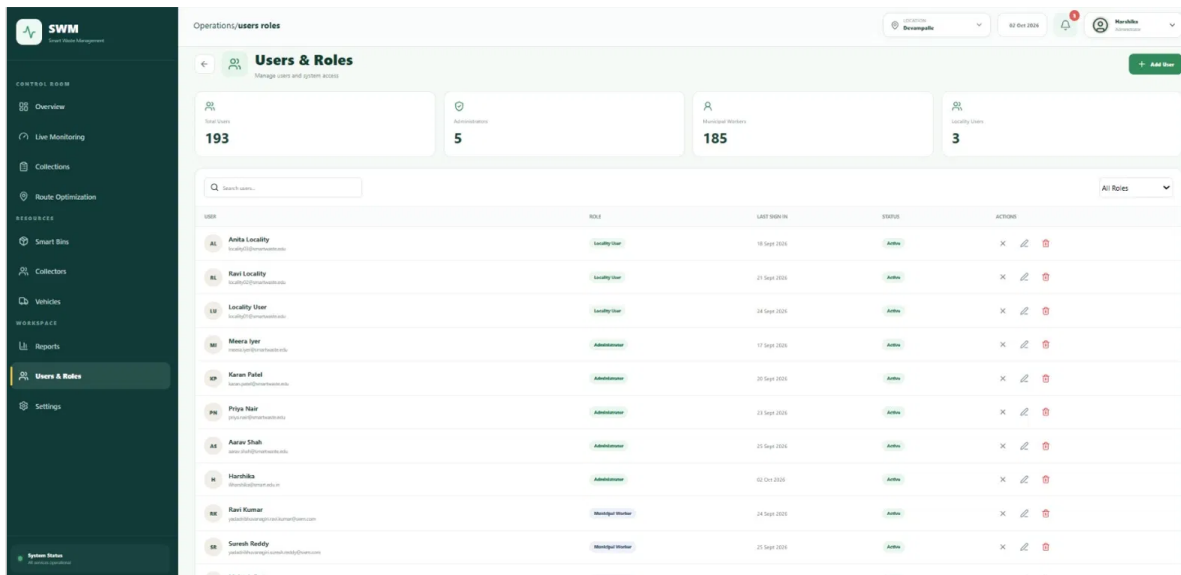


Figure A.21 - Users & Roles

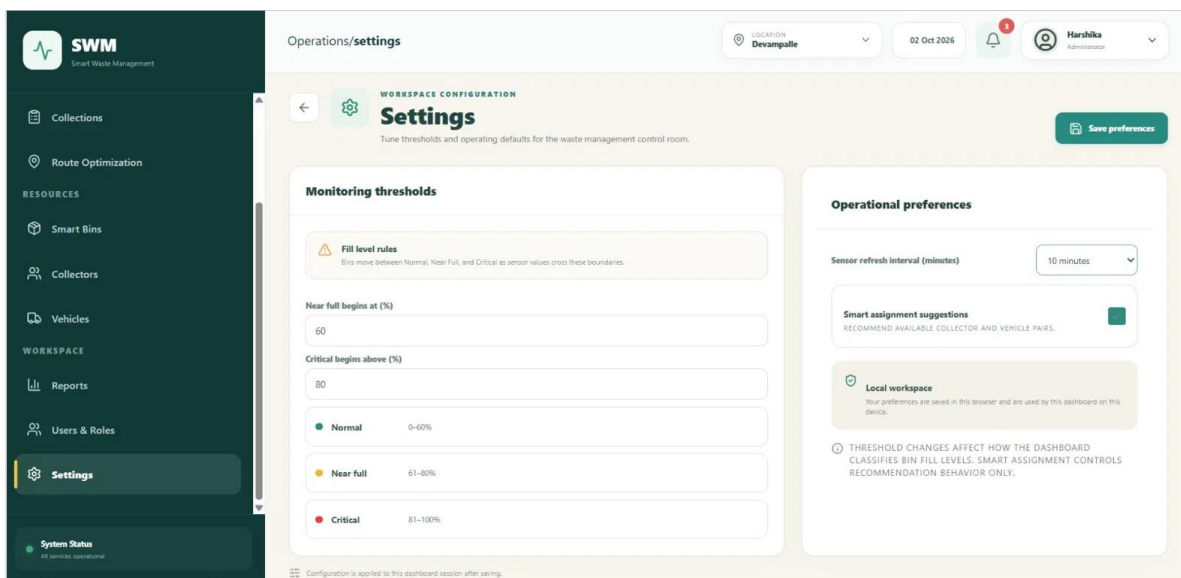


Figure A.22 - Settings

Sample Code Snippets

The following excerpts show the essential parts of the implementation.

A. Database schema (excerpt)

```
CREATE TABLE bins (
  bin_id          SERIAL PRIMARY KEY,
  bin_code        VARCHAR(20) UNIQUE NOT NULL,
  location_name   VARCHAR(100) NOT NULL,
  area            VARCHAR(60),
  capacity_litres INT NOT NULL CHECK (capacity_litres > 0),
  threshold_percent INT NOT NULL DEFAULT 80
  CHECK (threshold_percent BETWEEN 1 AND 100),
  status          VARCHAR(20) NOT NULL DEFAULT 'ACTIVE'
);

CREATE TABLE waste_levels (
  reading_id      BIGSERIAL PRIMARY KEY,
  bin_id          INT NOT NULL REFERENCES bins(bin_id),
```

```

    fill_level_percent NUMERIC(5,2) NOT NULL
                        CHECK (fill_level_percent BETWEEN 0 AND 100),
    recorded_at         TIMESTAMP NOT NULL DEFAULT NOW()
);
CREATE INDEX idx_wl_bin_time ON waste_levels (bin_id, recorded_at);

CREATE TABLE collection_requests (
  request_id          SERIAL PRIMARY KEY,
  bin_id              INT NOT NULL REFERENCES bins(bin_id),
  collector_id        INT REFERENCES collectors(collector_id),
  vehicle_id          INT REFERENCES vehicles(vehicle_id),
  triggered_level      NUMERIC(5,2) NOT NULL,
  status              VARCHAR(20) NOT NULL DEFAULT 'PENDING'
                        CHECK (status IN ('PENDING', 'ASSIGNED',
                                          'IN_PROGRESS', 'COMPLETED')),
  created_at          TIMESTAMP NOT NULL DEFAULT NOW(),
  assigned_at         TIMESTAMP,
  completed_at        TIMESTAMP
);

```

B. Storing a reading and detecting the threshold (Express.js controller)

```

exports.addReading = async (req, res) => {
  const { bin_id, fill_level_percent } = req.body;
  if (fill_level_percent < 0 || fill_level_percent > 100)
    return res.status(400).json({ error: 'Level must be 0-100' });

  const client = await pool.connect();
  try {
    await client.query('BEGIN');
    const bin = await client.query(
      "SELECT threshold_percent FROM bins " +
      "WHERE bin_id=$1 AND status='ACTIVE'", [bin_id]);
    if (bin.rowCount === 0) {
      await client.query('ROLLBACK');
      return res.status(404).json({ error: 'Bin not found' });
    }
    await client.query(
      'INSERT INTO waste_levels (bin_id, fill_level_percent) VALUES ($1,$2)',
      [bin_id, fill_level_percent]);

    if (fill_level_percent >= bin.rows[0].threshold_percent) {
      await client.query(
        `INSERT INTO collection_requests (bin_id, triggered_level, status)
        SELECT $1::int, $2::numeric, 'PENDING'
        WHERE NOT EXISTS (SELECT 1 FROM collection_requests
                          WHERE bin_id = $1::int
                          AND status IN ('PENDING', 'ASSIGNED', 'IN_PROGRESS'))`,
        [bin_id, fill_level_percent]);
    }
    await client.query('COMMIT');
    res.status(201).json({ message: 'Reading stored' });
  } catch (err) {
    await client.query('ROLLBACK');
    res.status(500).json({ error: err.message });
  } finally {
    client.release();
  }
};

```

C. Data simulator (excerpt)

```

const levels = {};
async function tick(bins) {
  for (const bin of bins) {
    const start = levels[bin.bin_id] ?? Math.random() * 40;
    const rise = Math.random() * bin.rate; // each bin fills at its own speed
    levels[bin.bin_id] = Math.min(100, start + rise);
  }
}

```

```

    await fetch(`${process.env.API_URL}/api/waste-levels`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json',
        Authorization: `Bearer ${process.env.SIM_TOKEN}` },
      body: JSON.stringify({
        bin_id: bin.bin_id,
        fill_level_percent: Number(levels[bin.bin_id].toFixed(2)),
      }),
    });
  }
}
setInterval(() => tick(bins), Number(process.env.SIM_INTERVAL_MS) || 5000);

```

Note: the code above is an illustrative excerpt of the logic described in this report. Replace it with the exact code from your project files if they differ.

Installation / Setup Instructions

1. Install **Docker Desktop**, **Visual Studio Code** and **Postman** on your computer.
2. Download the project folder (or clone the repository) and open it in Visual Studio Code.
3. Create a `.env` file in the project root and set the database and application values listed in Section 9.2.
4. Open the VS Code terminal and run: `docker compose up --build`.
5. Wait until the logs show that PostgreSQL is ready and the API is listening on port 3000.
6. Open Postman, import the project collection and set the `baseUrl` variable to `http://localhost:3000`.
7. Run the register and login requests, then add bins and watch the readings appear.
8. To stop everything, press `Ctrl + C` or run `docker compose down`.

User Manual / Guide

1. **Register and log in.** Send the register request with name, email and password, then the login request. Copy the token from the response; Postman uses it for later requests.
2. **Add bins (admin).** Send the add-bin request with the bin code, location, area, capacity and threshold. Use the list-bins request to confirm that the bin was saved.
3. **Add collectors and vehicles (admin).** Register each collector and each vehicle so that they can be assigned later.
4. **Watch monitoring.** The simulator starts automatically with the containers. Use the latest-levels request to see the current fill level of every bin, or the bin-history request for one bin.
5. **View collection requests.** When a bin crosses its threshold, a PENDING request is created. List requests with `status=PENDING`.

6. **Assign a request (supervisor).** Choose an available collector and vehicle and send the assign request.
7. **Update progress (collector).** Set the status to IN_PROGRESS when starting and COMPLETED when the bin is emptied.
8. **View reports.** Use the report requests to see waste-level history, collection statistics, and bin-wise or location-wise analysis.

Geo Tag Photos with Guide

Review 1



Review 2

